NeuroOpt: Throughput-aware peephole optimization with a learned cost oracle

24BCE2451 - Zayan Mohammed

24BCE2472 - Prabhjot Singh

24BCE2612 - Ayushmaan Nayak

24BCB0070 - Dhruva Shiraguppi

OVERVIEW

Contents

01 **Problem Identification**
Legality versus profitability; why current methods fall short; objectives

02 **Literature Survey**
Ten sources in four groups, and the research gap they leave

03 **Novelty**
What is not new, what NeuroOpt contributes, and how to word the claim

04 **Implementation**
Pipeline, selector, rewrite rules, verification, evaluation plan, roadmap

05 **Summary**
The research question, what is not yet claimed, and what the project will establish

01

Problem Identification

PROBLEM IDENTIFICATION

Many encodings, one meaning

SAME RESULT BITS, DIFFERENT COST

Zero a register	`mov rax, 0`	→ `xor rax, rax`	7 bytes → 3 bytes
Multiply by 8	`imul rbx, rbx, 8`	→ `shl rbx, 3`	multiplier → shifter
Copy to itself	`mov rcx, rcx`	→ `(delete)`	no effect on a 64-bit register

Classic peephole passes use hand-written rules and a fixed cost per instruction. The weak point is the word “cheaper”: who decides, and on what evidence.

PROBLEM IDENTIFICATION

Two separate questions

Legality

Does the rewrite preserve registers, memory and flags? It must be exact: one wrong rewrite breaks the program.

Profitability

Among legal options, which runs fastest? It depends on dependencies, ports and scheduling. A wrong guess costs speed only.

LEGALITY TRAPS

`imul rbx,rbx,8 → shl rbx,3`

Flags differ, so legal only if they are dead

`mov rax,0 → xor rax,rax`

xor clobbers CF, which a later adc may read

`remove mov eax,eax`

A 32-bit write zeroes the upper half of RAX

PROBLEM IDENTIFICATION

Why current methods fall short

1

Static cost tables

Score each instruction alone, so they miss overlap and port pressure across a sequence.

2

Analytical models

llvm-mca and uops.info are accurate for what they model, but built per microarchitecture.

3

Learned predictors

Ithemal and GRANITE are judged as predictors, never inside a correctness-gated rewriter.

Research question Does better cost prediction lead to better optimization decisions?

PROBLEM IDENTIFICATION

Problem statement

$$B^* = \operatorname{argmin} \hat{c}(B') \quad \text{over } B' \in C(B) \cup \{B\}$$

$$\text{subject to} \quad \hat{c}(B) - \hat{c}(B') > \tau$$

B a basic block

C(B) candidates from correctness-gated rewrite rules

ĉ cost oracle: static proxy now, learned LSTM later

τ confidence margin; if no candidate clears it, keep B

The question is whether a learned ĉ, trained on measured throughput, yields better choices than a static table, and whether that holds under measurement and not only on held-out prediction error.

PROBLEM IDENTIFICATION

Four objectives, two delivered

1 Represent assembly — DONE

Decode x86-64 blocks into consistent, model-ready tokens.

2 Generate legal rewrites — DONE

Emit candidates only when operand and flag-safety guards hold.

3 Learn performance — PLANNED

Train an embedding + LSTM regressor on measured BHive throughput.

4 Test decisions — PLANNED

Compare choices with the static table and llvm-mca; verify on hardware.

SUCCESS MEASURES

Prediction MAPE, Spearman rank correlation

Decisions Blocks improved, geometric-mean gain

Safety No illegal rewrite accepted

02

Literature Survey

LITERATURE SURVEY

Peephole optimization

McKeeman

Peephole Optimization · CACM 8(7), 1965

Introduced the term. Inspect a small window of generated code and discard redundant instructions in the final compilation stage.

We take small, local, verifiable rewrites.

Limit “better” means fewer or simpler instructions.

Aho, Lam, Sethi, Ullman

Compilers, 2nd ed., §8.7 · 2006

Textbook treatment: a sliding window over target code. Rule families: redundant loads and stores, unreachable code, flow of control, strength reduction, machine idioms.

We take the rule vocabulary (imul to shl, mov 0 to xor).

Limit hand-written rules and fixed costs.

Common thread small, local rewrites chosen by hand-written rules.

LITERATURE SURVEY

Superoptimization

Massalin

Superoptimizer · ASPLOS II, 1987

Finds the shortest program for a function by exhaustive search, filtered by a probabilistic test. Reported 12-instruction programs after hours on a 16 MHz 68020.

We take candidate generation and checking are separate from choosing.

Limit objective is length; does not scale to compiler use.

Bansal and Aiken

Peephole Superoptimizers · ASPLOS XII, 2006

Learns rules offline from training binaries; equivalence by tests, then SAT. Up to 10x on kernels over gcc, 1 to 5% on SPEC. Ranks candidates with a hand-built cost; counters were too noisy.

We take a rule database; selection needs a cost function.

Limit static cost, not studied separately. Legality is stronger than ours.

Common thread search finds candidates, but the cost that ranks them is still hand-built.

LITERATURE SURVEY

Performance models

llvm-mca

LLVM Project · command guide

Static analyzer: estimates instructions per cycle and resource pressure for a given CPU from LLVM scheduling models.

We take a reproducible baseline.

Limit accuracy is bounded by the model; BHive reports over 30% deviation for models like IACA and llvm-mca.

uops.info

Abel and Reineke · ASPLOS 2019

Auto-generated microbenchmarks give latency, throughput and port usage per instruction. Nehalem to Coffee Lake in the paper; more CPUs on the site.

We take ground truth for single instructions.

Limit per instruction, not per block; interactions are left to the consumer.

Common thread accurate for what they model, but not learned from measured blocks.

LITERATURE SURVEY

Throughput predictors

Ithemal

Mendis et al. · ICML 2019

Hierarchical LSTM over opcodes and operands predicts block throughput. Under half the error of llvm-mca and IACA; portable across microarchitectures.

We take the model family and regression framing; no novelty claimed.

Limit evaluated as a predictor, not inside a rewriter.

GRANITE

Sykora et al. · IISWC 2022

Graph neural network over structural and data dependencies. 6.9% average error, 1.7 points better than prior best, about 3x faster training and inference, multi-task.

We take dependency-aware features; a later comparison model.

Limit predicts cost; does not choose rewrites.

Common thread strong predictors, never placed inside a correctness-gated rewriter.

LITERATURE SURVEY

Measured data and learned decisions

BHive

Chen et al. · IISWC 2019

Benchmark suite and profiler measuring throughput of arbitrary blocks; clusters by resource use; evaluated IACA, llvm-mca, Ithemal and OSACA.

We take measured labels and the measurement method.

Limit blocks are real code, not rewrites; labels are per microarchitecture.

MLGO

Trofin et al. · arXiv 2021

Framework replacing LLVM heuristics with learned policies. Inlining for size up to 7% smaller than -Oz; register allocation gained 0.3 to 1.5% in queries per second.

We take learned decisions are practical; semantics-preserving choices.

Limit different decisions; not a peephole selector.

Common thread measured labels exist and learned decisions work in LLVM, but not for peephole selection.

LITERATURE SURVEY

Research gap

Work	Rewrites code	Correctness check	Learned cost	Choice evaluated
Classical peephole	Yes	Rules	No	No
Superoptimizers	Yes	Tests, SAT	No	End-to-end only
llvm-mca, uops.info	No	n/a	No	n/a
Ithemal, GRANITE	No	n/a	Yes	Predictor only
MLGO	Other decisions	n/a	Yes	Yes
NeuroOpt	**Yes**	**Guards, flag liveness**	**Planned**	**Planned**

Gap Among the works surveyed, none combines guarded rewriting, a learned oracle and decision-level evaluation.

03

Novelty

NOVELTY

A decision layer, not a new model

NOT NEW

- Peephole rewrite rules
- LSTM throughput prediction (Ithemal)
- Measured block data (BHive)
- ML-guided compilation (MLGO)
- SAT-verified rule generation

WHAT NEUROOPT CONTRIBUTES

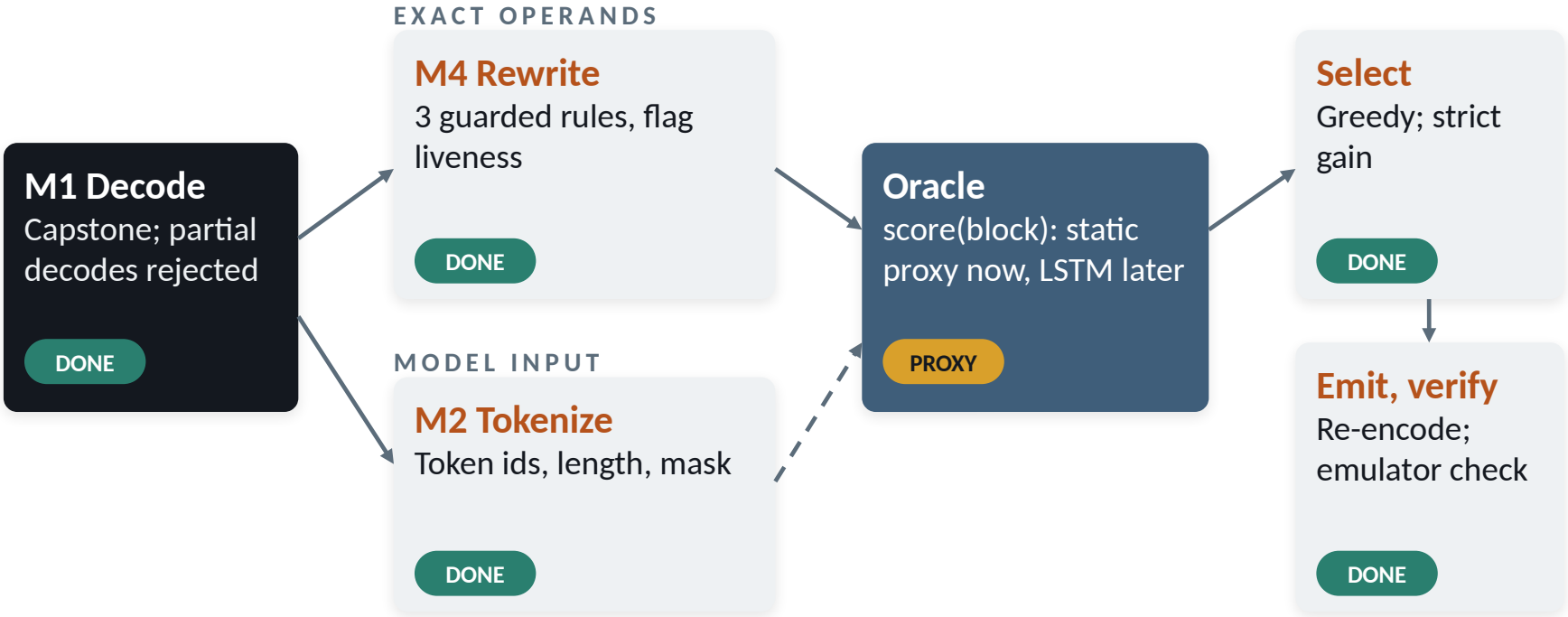
1. **Separation of concerns**
   Symbolic guards own legality; the model only ranks legal options.
2. **Integrated pipeline**
   Decode, rewrite, score, select and verify behind one swappable oracle.
3. **Decision-level evaluation**
   Win rate, false positives and geometric-mean gain, not only prediction error.
4. **Honest, reproducible scope**
   A tested subset; unsupported code is left untouched.

04

Implementation

IMPLEMENTATION

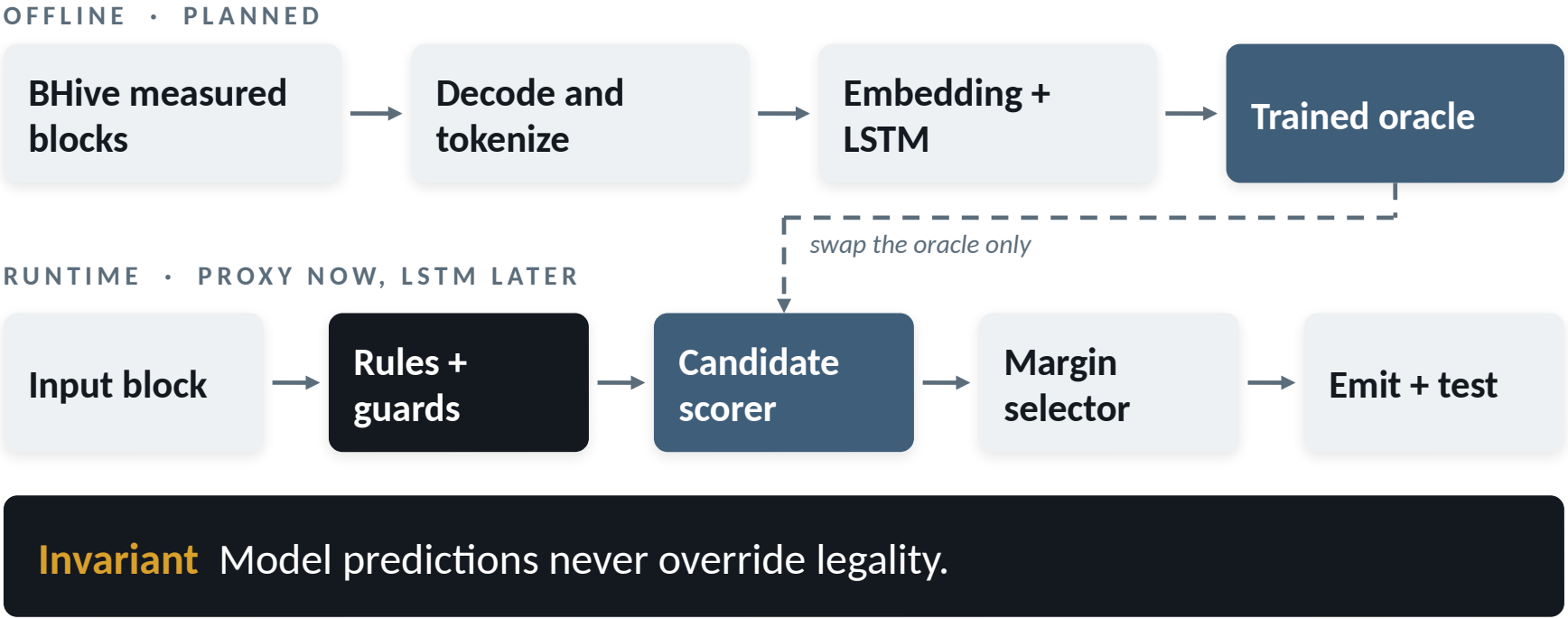
Pipeline



Dashed link: the learned LSTM is not built yet. Legality never depends on the model; the oracle only ranks candidates that are already legal.

IMPLEMENTATION

Offline training, runtime selection



Invariant Model predictions never override legality.

IMPLEMENTATION

Rewrite rules and flag liveness

Rule	Rewrite	Guard
R1	`mov r,0 → xor r,r`	All six flags dead afterwards
R2	`imul r,r,2^k → shl r,k`	All six flags dead afterwards
R3	`mov r64,r64 → delete`	64-bit only; 32-bit write zero-extends

BACKWARD FLAG LIVENESS

```
live_before = reads ∪ (live_after − kills)
```

A rewrite is allowed only if no flag it may change is live afterwards.

CONSERVATIVE CHOICES

- Flags live at block exit
- Jumps and calls read all flags
- Shift by CL kills nothing
- Unknown instructions read all flags

IMPLEMENTATION

How the selector chooses a rewrite

1. Decode B; reject unsupported blocks
2. Enumerate rules with live-flag guards
3. Score B and every legal B′
4. Choose the argmin; require gain > τ
5. Emit and run the verification harness

SCORING MODES

Baseline

Fixed per-opcode proxy. This is the oracle that runs today.

Proposed

LSTM trained on measured BHive throughput. Planned.

Margin τ Gains below τ keep the original block.

IMPLEMENTATION

Worked example and verification

ORIGINAL · 4

```
mov rax, 0
imul rbx, rbx, 8
mov rcx, rcx
add rax, rbx
```

SELECTED · 3

```
xor rax, rax
shl rbx, 3
add rax, rbx
```

6.00 → 2.25

Static proxy score, 62.5% lower. A proxy for testing the selector, not a hardware speedup. The final add overwrites every flag, so both flag-changing rewrites are legal.

17 / 17

tests pass, including fuzzing

0 of 3,488

illegal rewrites in fuzzing with guards on

894 of 1,690

caught with guards off: the harness can detect bugs

IMPLEMENTATION

A data contract for aligned labels

PROPOSED TRAINING RECORD

`block_id`	stable id / hash
`hex_bytes`	machine-code bytes
`instruction_tokens`	opcode + operand classes
`cpu_arch`	target microarchitecture
`target_y`	measured cycles per block
`split`	train / validation / test

A planned normalized manifest, not an existing BHive schema.

DATA HYGIENE

- Exclude invalid or partially decoded blocks
- Prevent identical or near-duplicate train–test leakage
- Keep the CPU label and units explicit

Eval split Hold out whole program families where feasible.

IMPLEMENTATION

How the hypothesis will be tested

Level	Measure	Protocol
Correctness	Equivalent outputs and flags	Randomized states; adversarial live-flag cases
Prediction	MAPE, MAE, Spearman	BHive held-out blocks; CPU-matched labels
Selection	Win rate; regret vs best candidate	Same candidate pool: static vs LSTM vs llvm-mca
End-to-end	Geometric-mean throughput gain	Physical timing, repeated runs, variance reported

ABLATIONS

- Guards on / off
- LSTM vs fixed table
- Margin τ
- Tokenization
- Training-set size

Rule Hold candidates fixed; change only the ranking.

IMPLEMENTATION

What can invalidate a claimed gain

Risk	Control
Noisy hardware timings	Repeat measurements; report confidence and variance
Train–test block leakage	Deduplicate; use source-disjoint splits when feasible
Wrong CPU generalization	Train and evaluate on a specified microarchitecture
Over-aggressive rewriting	Reject unsupported operands; keep the legal original
Compilation-time overhead	Measure model inference cost against time gained

Scope boundary Three rule families; six arithmetic flags; no formal proof or hardware gain yet.

IMPLEMENTATION

Limits and roadmap

KNOWN LIMITS

- Static proxy oracle; no trained model or hardware timing yet
- Three rules; general-purpose 32 and 64-bit registers
- Guards are tested, not formally proven
- Only the six arithmetic flags are tracked
- Per basic block; flags assumed live at exit
- Greedy selection, not optimal search

ROADMAP

M0 — **Static prototype**
Repeatable tests, runnable demo

M1 — **BHive preparation**
Train, validation, test manifest

M2 — **LSTM training**
Held-out MAPE, rank correlation

M3 — **Integration**
Decision win rate, geometric-mean gain

05

Summary

SUMMARY

One question, one experiment

Research question Does a learned block-throughput model select better legal rewrites than a fixed-cost table?

CURRENT PROTOTYPE

Decode → Rewrite → Guard → Proxy-select → Verify

Static proxy oracle; 17 of 17 tests pass.

NOT YET CLAIMED

- Trained LSTM
- Decision-level hardware gains
- Formal verification

SUCCESS CRITERION

Better measured choices, not merely lower prediction error

SUMMARY

What this project will establish

Central contribution A correctness-gated peephole optimizer with a replaceable cost oracle, evaluated by the quality of its actual rewrite choices.

DONE

Built

Decode, safe candidate rules, proxy selection, tests

PLANNED

Next

Train on BHive; score candidates with the LSTM

PLANNED

Prove

Use hardware timing to test whether decision quality improves

References

[1] W. M. McKeeman, “Peephole optimization,” *Commun. ACM*, vol. 8, no. 7, pp. 443–444, Jul. 1965.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[3] H. Massalin, “Superoptimizer: A look at the smallest program,” in *Proc. 2nd Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS II)*, Palo Alto, CA, USA, 1987, pp. 122–126.

[4] S. Bansal and A. Aiken, “Automatic generation of peephole superoptimizers,” in *Proc. 12th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS XII)*, San Jose, CA, USA, 2006, pp. 394–403.

[5] LLVM Project, “llvm-mca - LLVM Machine Code Analyzer,” *LLVM Documentation*. [Online]. Available: https://llvm.org/docs/CommandGuide/llvm-mca.html. Accessed: Oct. 10, 2026.

[6] A. Abel and J. Reineke, “uops.info: Characterizing latency, throughput, and port usage of instructions on Intel microarchitectures,” in *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2019, pp. 673–686.

[7] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin, “Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks,” in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 97, 2019, pp. 4505–4515.

[8] O. Sykora, P. M. Phothilimthana, C. Mendis, and A. Yazdanbakhsh, “GRANITE: A graph neural network model for basic block throughput estimation,” in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2022.

[9] Y. Chen *et al.*, “BHive: A benchmark suite and measurement framework for validating x86-64 basic block performance models,” in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2019.

[10] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, “MLGO: A machine learning guided compiler optimizations framework,” 2021, *arXiv:2101.04808*.